

2.3 - Quando o monólito mostra limites

Sinais reais e falsos alarmes

20 minutos para corrigir, 2 horas para publicar

- **Bug crítico no módulo de notificação dos clientes**
 - Correção pronta em 20 minutos, deploy travado por horas

LUNALOG

A LunaLog encontrou um bug crítico: os clientes recebiam alertas de entrega errados. Ana identificou e corrigiu a falha em vinte minutos. O problema veio depois. Publicar a correção exigia o build completo do monólito, rodar os 340 testes e subir o sistema inteiro. Foram duas horas até a correção chegar em produção, e os clientes seguiram recebendo alertas errados o tempo todo. Esse foi o sinal mais claro de que o monólito começou a custar mais do que entregava.

Sinais reais de que o monólito limita

- **Qualquer mudança exige rebuild e restart total**
 - O ciclo de deploy fica lento e arriscado
- **Times diferentes colidem no mesmo código**
 - Conflitos e filas de merge atrasam todo mundo
- **Um componente sob pico derruba o sistema inteiro**
 - *Não há isolamento de falha entre áreas distintas*
- **A base cresceu além da compreensão de uma pessoa**
 - Ninguém mais tem o mapa mental completo do sistema

Sinais reais vs. falsos alarmes

Sinais reais de limite

- Deploy lento bloqueia correções urgentes
- Escalar uma parte obriga a escalar tudo
- Fronteiras de time batem no mesmo código
- Falha localizada vira falha global

Falsos alarmes para migrar

- Microsserviços estão na moda agora
- Um artigo disse que monólito não escala
- A equipe quer aprender uma tecnologia nova
- O concorrente usa arquitetura distribuída

Migre por dor, não por moda

- **Migrar cedo demais multiplica custo e risco**
- **A dor real justifica a complexidade adicional**

Microserviços resolvem problemas de escala e autonomia de times. Se a sua dor não é nenhuma dessas, dividir o sistema cria problemas novos sem resolver os antigos.

- Documente o sinal concreto antes de decidir migrar



O monólito chegou ao limite, e a resposta automática do mercado é uma só: microserviços. Mas microserviços de verdade vão muito além de quebrar o sistema em pedaços menores. Existem princípios que fazem o estilo funcionar, e ignorá-los custa caro. O que separa um microserviço de um pedaço de código que só parece um?

Microserviços: princípios e bounded contexts